

ECS 140A Programming Languages

Homework 5

About This Assignment

- This assignment asks you to complete programming tasks using the Go programming language.
- **You are only allowed to use the subset of Go that we have discussed in class. No credit will be given in this assignment if any of the problem solutions use material not discussed in class.** Please use Piazza for any clarifications regarding this issue.
 - Especially, the use of the `runtime` and `reflect` packages is forbidden.
 - Although the package `time` is discussed in class, you are NOT allowed to use it in this homework (e.g. `time.Sleep`).
- **Since the focus of this homework is concurrency, homework solutions are required to be concurrent in order to receive credit.**
- To complete the assignment (i) download `hw5-handout.zip` from Canvas, (ii) modify the `members.txt` and `.go` files in the `hw5-handout` directory as per the instructions in this document, and (iii) zip the `hw5-handout` directory into `hw5-handout.zip` and upload this zip file to Canvas by the due date.

Do not change the file names, create new files, or change the directory structure of `hw5-handout`.
- This assignment can be worked on in a group. Even if working in a group, each student has to submit a copy of the work to Canvas to get credit. See the Syllabus for more details.
- List all the names and email addresses of all members of the group in the `members.txt` file in the `hw5-handout` directory, one per line in the format `name <email>`.

If you are working individually, then only add your name and email to `members.txt`.
- Refer to **Homework 0** for instructions on installing the correct versions of the programming language as well as using CSIF computers.
- Begin working on the homework early.
- Apart from the description in this document, look at the unit tests provided to understand the requirements for the code you have to write.
- Post questions on Piazza if you require any further clarifications. Use private posts if your question contains part of the solution to the homework.
- Keep your homework solution after you submit it. You may need to use it for later assignments.

1 Bug1 (10 points)

- Modify code in `hw5-handout/bug1/bug1.go` to fix the concurrency bug.
- Some unit tests are provided for you in `hw5-handout/bug1/bug1_test.go`. **You are not allowed to make changes in `hw5-handout/bug1/bug1_test.go` to fix the bug.**

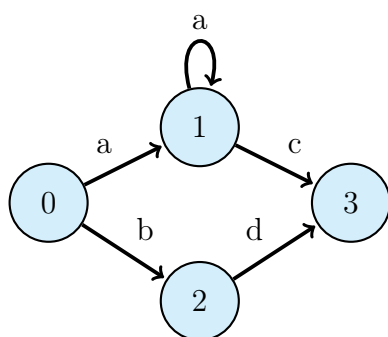
From the `hw5-handout/bug1` directory, run the `go test` command to run the unit tests.

Run the `go test -race` command to run the unit tests with the [race detector](https://blog.golang.org/race-detector),¹ a tool for finding race conditions in Go code.

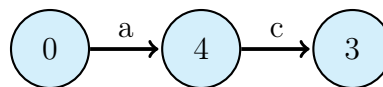
- From the `hw5-handout/bug1` directory, run `go test -cover` command to see the current code coverage.
- If needed, add more unit tests in `hw5-handout/bug1/bug1_test.go` to get 100% code coverage for the code in `hw5-handout/bug1/bug1.go`.
- From the `hw5-handout/bug1` directory, run the following two commands to see which statements are covered by the unit tests:

```
$ go test -coverprofile=temp.cov
$ go tool cover -html=temp.cov
```

2 lgraph (25 points)



(a) Labeled directed graph G_1



(b) Labeled directed graph G_2

Figure 1

For this part of the assignment, you only need to modify `hw5-handout/graph/lgraph.go` and `hw5-handout/graph/lgraph_test.go`.

- A *labeled directed graph* G is a directed graph where each edge is labeled with a symbol.

Figure 1 shows two examples of labeled directed graphs G_1 and G_2 .

¹<https://blog.golang.org/race-detector>

- $[\ell_1, \ell_2, \dots, \ell_k]$ is said to be a *sequence* from nodes u to v in G if there exists a path from u to v in G whose edges are labeled with $\ell_1, \ell_2, \dots, \ell_k$ in order. The number of labels k in the sequence is defined as the *length of the sequence*.

The sequences $[a, c]$ and $[b, d]$ are both sequences of length 2 from node 0 to node 3 in graph G_1 . The sequence $[a, a, c]$ is a sequence of length 3 from node 0 to node 3 in graph G_1 .

The sequence $[a, c]$ is a sequence of length 2 from node 0 to node 3 in graph G_2 .

- In `hw5-handout/lgraph/lgraph.go`, write a **concurrent** implementation of the `FindSequence` function.

`FindSequence(g1, g2, s, t, k)` returns `(S, true)` if there is a sequence S of length k from node `s` to node `t` in graph `g1` and S is *not* a sequence from `s` to `t` in graph `g2`; else it returns `nil, false`.

The goal of this assignment is to test your knowledge on concurrency in Go, thus no credit will be given if your implementation is not concurrent.

- Consider the graphs G_1 and G_2 shown in Figure 1:

`FindSequence(G1, G2, 0, 3, 2)` should return `([b, d], true)`.

`FindSequence(G1, G2, 0, 3, 3)` should return `([a, a, c], true)`.

`FindSequence(G1, G2, 3, 0, 2)` should return `(nil, false)`.

`FindSequence(G2, G1, 0, 3, 2)` should return `(nil, false)`.

- You can add additional helper functions in `hw5-handout/lgraph/lgraph.go`, if needed.
- If needed, write new tests, in `hw5-handout/lgraph/lgraph_test.go` to ensure that you get 100% code coverage for your code.

From the `hw5-handout/lgraph` directory, run the `go test -cover` command to see the current code coverage.

From the `hw5-handout/lgraph` directory, run the following two commands to see which statements are covered by the unit tests:

```
$ go test -coverprofile=temp.cov
$ go tool cover -html=temp.cov
```

- Implementing a sequential version of `FindSequence` will get you no points.